Lab 6 Inode identity and recursive pathname search

Tacu Igor, FAF 242
Operating Systems, Technical University of Moldova
New xv6 reproduction on 9 November 2026

Scope and implementation

This experiment examines two different tasks: reading metadata for a supplied name and walking a directory tree to discover names. The complete implementations are included as `xv6/user/mystat.c` and `xv6/user/find.c`, together with their UPROGS entries and the full kernel tree needed to build them. The following results were collected from that tree in QEMU with one emulated hart, using a fresh generated filesystem image.

The required README/r2 sequence was run first, including removing README and reading r2. README was then restored through a hard link so later trace tests could use it. An additional disposable file named specimen checks the behavior on a short payload. The test tree contains both file and directory matches.

Part 1 Types and empty directories

Types 1, 2 and 3 mean directory, regular file and device. Root has inode 1; dot and dot-dot both identify it because root is its own parent. A new directory a occupied 32 bytes, representing two 16-byte directory entries: a two-byte inode number and fourteen-byte name, for dot and dot-dot. The public stat structure exposes dev, ino, type, nlink and size.

```
$ mkdir a
$ ls a
.              1 35 32
..             1 1 1024
```

Reading metadata without confusing a name with an inode

`mystat` loops over its operands and calls `stat` for each one. It translates the three public xv6 types into file, directory, and device labels, and prints the inode number, link count, and byte size. If one lookup fails, it records failure and continues through later operands instead of aborting at the first bad name. The size uses the format for a 64-bit unsigned value rather than treating the field as a small integer.

README initially reported inode 2, one link and 2441 bytes. After ln, README and r2 shared inode 2 and two links. After removing README, r2 retained inode 2 and size 2441 with one link, and cat could still read its contents. The extra ten-byte specimen received inode 40 in this image and showed the same link behavior.

```
$ mystat README . console
README: type=file inode=2 links=1 bytes=2441
.: type=directory inode=1 links=4 bytes=1024
console: type=device inode=30 links=1 bytes=0
$ ln README r2
$ mystat README r2
README: type=file inode=2 links=2 bytes=2441
r2: type=file inode=2 links=2 bytes=2441
$ rm README
$ mystat r2
r2: type=file inode=2 links=1 bytes=2441
$ cat r2 | wc
48 336 2441


$ echo igor-file > specimen
$ mystat specimen
specimen: type=file inode=40 links=1 bytes=10
$ ln specimen alias
$ mystat specimen alias
specimen: type=file inode=40 links=2 bytes=10
alias: type=file inode=40 links=2 bytes=10
$ rm specimen
$ mystat alias
alias: type=file inode=40 links=1 bytes=10
$ cat alias
igor-file
$ mystat missing-path
mystat: cannot stat missing-path
```

Inode numbers depend on allocation order. The tested invariant is that both names identify one inode and that removing one name preserves the remaining link and contents.

Part 3 Measuring the limits

BSIZE is 1024, NDIRECT is 12, NINDIRECT is 1024 divided by a four-byte block number, and DIRSIZ is 14. Maximum file payload is `(12 + 1024/4) * 1024 = 274432` bytes, or 268 KiB. The indirect block is metadata, not part of that payload.

Creating averyveryverylongname stored the fourteen-character name averyveryveryl. The builder reported 2000 blocks: 47 metadata and 1953 data. Metadata comprises two boot/super blocks, 31 log blocks including its header, 13 inode blocks and one bitmap block.

```
42:#define IPB (BSIZE / sizeof(struct dinode))
5:#define BSIZE   1024 // block size
26:#define NDIRECT   12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total
    2000
$ mkdir averyveryverylongname
$ ls
averyveryveryl 1 36 32
[Other listing entries are retained in the full recording.]
```

The on-disk dinode fields are type, major, minor, nlink, size and twelve direct addresses plus one indirect address. A name is stored in a directory entry, not in the inode. Major and minor identify a device for a device inode.

Part 4 A find implementation that also reports directories

The search utility takes a starting pathname and a basename. After stat it compares the current name before deciding whether to recurse, allowing directories themselves to match. It copies the fixed-width `dirent.name` into a DIRSIZ-plus-one buffer and adds a terminator rather than assuming the kernel entry is already a C string.

The loop skips unused entries, dot, and dot-dot. Before constructing a child path, it checks the parent, separator, name, and terminator against its 512-byte buffer. Lookup and read failures produce diagnostics and a failure status. Every opened directory is closed after traversal. Ancestor descriptors remain open during recursion, so very deep trees are limited by xv6's descriptor limit.

Tests that distinguish traversal from correct matching

The tree contains `tree/target` and `tree/branch/target`, plus a directory `tree/branch/targetdir`. Searching for `target` printed the two files; searching for `targetdir` printed the directory; searching for `branch` printed the intermediate directory. A fourteen-character filename checked the DIRSIZ boundary, and a missing starting path exercised the diagnostic path. Invoking `find` without the two operands printed usage rather than interpreting uninitialized arguments.

```
$ mkdir tree
$ mkdir tree/branch
$ echo root-copy > tree/target
$ echo nested-copy > tree/branch/target
$ mkdir tree/branch/targetdir
$ find tree target
tree/branch/target
tree/target
$ find tree targetdir
tree/branch/targetdir
$ find tree branch
tree/branch
$ echo edge > tree/abcdefghijklmn
$ find tree abcdefghijklmn
tree/abcdefghijklmn
$ find missing-path target
find: cannot stat missing-path
$ find
usage: find start basename
$
```

These tests would expose a common bug in which basename comparison is performed only for regular files. Such a version can descend into `branch` and still fail to print `branch` or `targetdir`. The directory-target assertions in `scripts/run-xv6.py` verify that behavior explicitly. The raw console output is retained in `Lab5-6/evidence/xv6.txt`, so the displayed paths can be checked against the commands that constructed them.

Why unlink does not always free the blocks immediately

In `kernel/sysfile.c`, the `ip->nlink--;` line in `sys_unlink` decrements the link count after removing the name. In `kernel/fs.c`, `iput` waits for zero links and the final relevant in-memory reference before `itrunc` frees data blocks. An open reference can therefore preserve an unlinked inode.

The experiment proves survival through another hard link. The open-but-unlinked explanation comes from source inspection; it is a separate lifetime case.

Running `scripts/run-xv6.py` rebuilds the image and checks these distinctive link and search results. A fresh generated disk prevents leftover test names from affecting a rerun.

Execution captures and code excerpts

The following screenshots show the read-only console of saved SSH execution records and source excerpts. They are linked to the raw recordings in the repository and retain an output hash for traceability. The complete source files accompany the report.

lab6-mystat

READ-ONLY EXECUTION RECORD · Ubuntu SSH · 9 November 2026 · xv6/user/mystat.c

Source excerpt

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(int argc, char **argv)
{
  if(argc < 2){
    fprintf(2, "usage: mystat path ...\n");
    exit(1);
  }
  int failed = 0;
  for(int i = 1; i < argc; i++){
    struct stat st;
    if(stat(argv[i], &st) < 0){
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      failed = 1;
      continue;
    }
    char *kind = st.type == T_DIR ? "directory" :
                 st.type == T_FILE ? "file" :
                 st.type == T_DEVICE ? "device" : "unknown";
    printf("%s: type=%s inode=%d links=%d bytes=%lu\n",
           argv[i], kind, st.ino, st.nlink, st.size);
  }
  exit(failed);
}
```

Recorded command output

```
$ mystat README . console
README: type=file inode=2 links=1 bytes=2441
.: type=directory inode=1 links=4 bytes=1024
console: type=device inode=30 links=1 bytes=0
$ ln README r2
$ mystat README r2
README: type=file inode=2 links=2 bytes=2441
r2: type=file inode=2 links=2 bytes=2441
$ rm README
$ mystat r2
r2: type=file inode=2 links=1 bytes=2441
$ cat r2 | wc
48 336 2441
```

Output SHA256 402bd1d0f86bc5666150e342

Figure 1: Metadata code and hard link execution



Figure 2: Recursive search code and required test tree